

MathBattle Simulator: A Method-Oriented Game with Command-Line Inputs

Submission Instructions

- Submit your source code file `MathBattle.java` to Google Classroom.
- Submit at least **3 screenshots**:
 - 2 showing valid game execution (with different command line arguments) with a declared winner.
 - One showing input validation error handling.

Purpose

This lab focuses on a deep understanding and practical application of:

- Static methods and method decomposition
- Method overloading
- Use of constants and `final` variables
- Argument passing via `String[] args`
- Random number generation via `SecureRandom`
- Use of `Math` class
- Enumerations (`enum`) for game state representation

Problem Description

You are to design and implement a command-line Java application that simulates a math-based game between two players. The application must read the following from the command line:

```
java MathBattle <Player1Name> <Player2Name> <Rounds>
```

Example:

```
java MathBattle Alice Bob 5
```

The program will simulate a number of rounds between the players. In each round:

- Both players roll a virtual 10-sided die (values 1–10) using `SecureRandom`.
- Each player is given a math question whose difficulty is determined by their die roll.
- The correct answer is computed internally.
- Each player's response is simulated using a guessing method (within ± 2 range).
- If the guessed value equals the actual result, that player earns 1 point.

After all rounds, the program must print:

- Final scores
- Player with the highest score
- Game result using an `enum` (WIN, LOSS, or DRAW)

Technical Requirements

- **You must use only command-line arguments.** Do not use `Scanner` or `BufferedReader`.
- Use at least **6 methods**, each with the following responsibilities:
 - `generateQuestion(int difficultyLevel)` **Purpose:** Generates a math question based on the provided difficulty level (easy, medium, hard), prints the question, and returns the correct answer. **Parameters:** `difficultyLevel` (1: Easy, 2: Medium, 3: Hard) **Returns:** An `int` representing the correct answer to the generated question.
 - `checkAnswer(int expected, int given)` **Purpose:** Checks whether the given answer is acceptable within a specified tolerance of the expected answer. **Parameters:** `expected` (correct answer), `given` (guessed answer) **Returns:** `true` if the guess is within `ERROR.TOLERANCE`, `false` otherwise.
 - `simulateGuess(int correctAnswer)` **Purpose:** Simulates a player's guess for a given correct answer by introducing random variation. **Parameters:** `correctAnswer` **Returns:** A guessed integer that may be close to the correct answer.
 - `simulateGuess(int correctAnswer, boolean bias)` (Overloaded) **Purpose:** Simulates a player's guess with optional bias. If `bias = true`, simulate a smarter or more accurate guess. **Parameters:** `correctAnswer`, `bias` flag **Returns:** A guessed integer based on whether bias is applied.
 - `printRoundResult(...)` **Purpose:** Displays all details for the round, including player rolls, questions, guessed answers, and whether each guess was correct. **Parameters:** Appropriate player and round data (e.g., player names, guesses, correctness) **Returns:** `void`
 - `printSummary(...)` **Purpose:** Displays the final scores and declares the result of the game using the predefined `enum`. **Parameters:** Player names and final scores **Returns:** `void`
- Use `SecureRandom` for random operations.
- Use static `final` constants for configuration:
 - `MAX_DICE_VALUE`: The maximum value a player can roll in a round (e.g., 10).
 - `MIN_DICE_VALUE`: The minimum value a player can roll in a round (e.g., 1).
 - `ERROR_TOLERANCE`: The maximum allowed deviation between the simulated guess and the actual result (e.g., ± 2).
- Use at least one method from the `Math` class (e.g., `Math.abs`, `Math.pow`, etc.).
- You must define and use an `enum` to represent the game result: WIN, LOSS, DRAW.

Connection Between Die Roll and Question Difficulty

To give functional meaning to the virtual die roll, you must use it to determine the difficulty level of the math question each player receives. Use the die value (1–10) to assign a difficulty level as follows:

- **Easy (Level 1):** Die roll is 1–3. Questions should use small integers (0-9) and basic operations (e.g., $2 + 3$, $4 - 1$).
- **Medium (Level 2):** Die roll is 4–7. Use larger numbers (0-20) and introduce multiplication (e.g., $15 * 3$, $15 - 9$).
- **Hard (Level 3):** Die roll is 8–10. Use 2-digit numbers greater than 20 and more complex expressions (e.g., $18 - 85$, $91 * 8$, $9/8$, $9\%8$).

You should implement a helper method:

```
public static int mapDieToDifficulty(int dieRoll) {  
    if (dieRoll <= 3) return 1;  
    else if (dieRoll <= 7) return 2;  
    else return 3;  
}
```

Then, based on this difficulty level, generate the appropriate math question using another method such as:

```
public static int generateQuestion(int difficultyLevel) {  
    // Generate operands and operation based on difficulty level  
    // Print the question  
    // Return the correct answer  
}
```

This connection ensures the game is dynamic, responsive to random inputs, and offers varied challenges to both players based on their die rolls.

Rules and Constraints

- Input validation is mandatory:
 - All three arguments must be present.
 - The number of rounds must be a positive integer.
- Use `break` and `continue` where appropriate.
- Use compound assignments and pre/post increment operators when applicable.

Important

Make sure your code is properly indented, commented, and compiles successfully. Late submissions will not be accepted.

Sample Output

```
MathBattle: Alice vs. Bob | Rounds: 3
```

```
[Round 1]
```

```
Alice rolls 7
```

```
Bob rolls 5
```

```
Alice's question: 3 + 6 = ?
```

```
Alice guessed: 9
```

```
Bob's question: 4 * 2 = ?
```

```
Bob guessed: 6
```

```
[Round 2]
```

```
...
```

```
== Final Scores ==
```

```
Alice: 2
```

```
Bob: 1
```

```
Result: Alice WINS!
```